

GPS SERVER

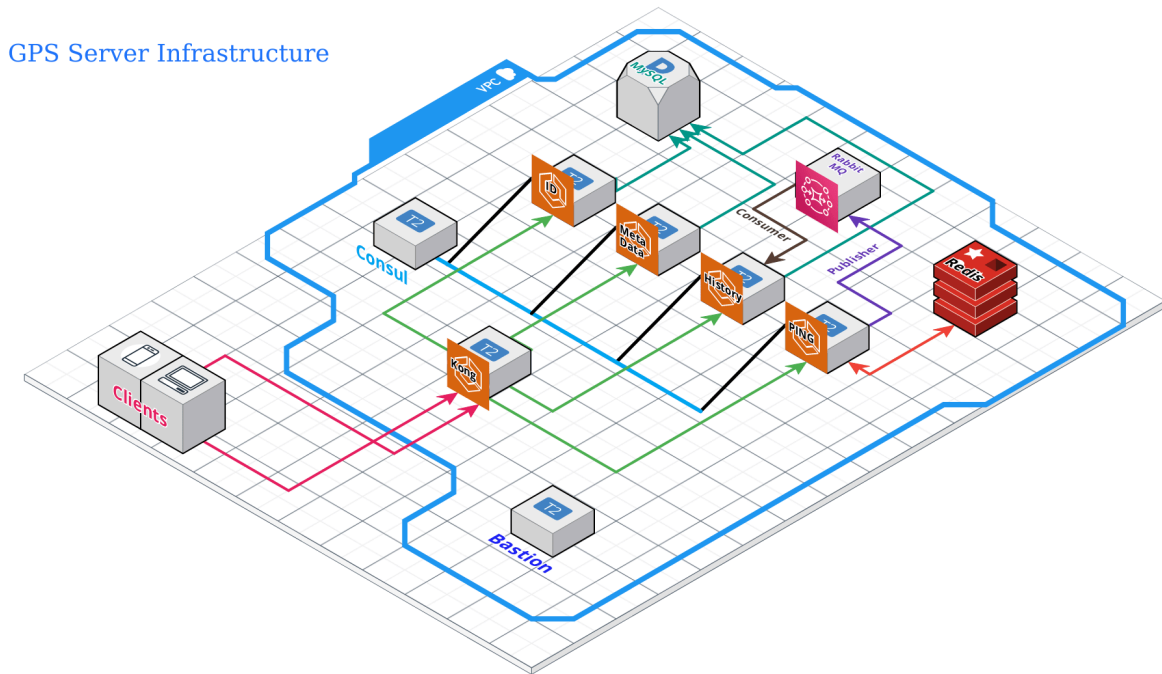


Figure 1: GPS Server Infrastructure

The figure is a bird's eye view of the GPS server infrastructure. There are four services in the GPS server and they are Ping, History, Id, and Metadata. Clients or users can only access the services through the Kong Gateway. There is an instance for configuration management named Consul. All the service has access to it. Before starting any service they get their configuration from the Consul instance. There is also a Rabbitmq for implementing message queue, Redis for cache/in-memory database, and a Mysql database to store data. Ping service is the publisher for Rabbitmq. All the user location data first goes to Ping service and Ping service pushes them into Rabbitmq and only the last location of each user is cached on Redis. History Service is the consumer of Rabbitmq. It consumes data from the message queue and stores them in the Mysql database. Id service is used for authentication of the user.

Services

Ping

The service saves all the locations of a user. At first, it stores the location data on memory and after a certain time, it pushes them to the Rabbitmq as a bulk. For every user, it stores the last location of users to the Redis cache. There are four endpoints in this service. They are

1. **GET /api/v1/ping**: This endpoint is for checking the health of the service.
2. **POST /api/v1/locations**: This endpoint saves all the user data.
3. **GET /api/v1/locations/users**: This endpoint is for retrieving all users within a specific radius.
4. **GET /api/v1/locations**: Get the last location of specific users.

History

The service provides all the location histories of a user. It **consumes** users' location data from **rabbitmq** and pushes them into the **database**. This service is the only consumer of the rabbitmq. There are two endpoints in this service. They are:

1. **GET /api/v1/ping**: This endpoint is for checking the health of the service.
2. **GET /api/v1/history**: The endpoint will provide all the location histories of a specific user.

Id

The service manages authentication and authorization. The service has five endpoints and they are:

1. **GET /api/v1/ping**: This endpoint is for checking the health of the service.
2. **GET /api/v1/internal/authenticate**: This endpoint is only for Gateway service's internal use.
3. **POST /api/v1/company/signup**: Endpoint to register a company. This is a restricted endpoint. This will not be provided to any person outside of the Realtime Location Service. All requests must have a sret header.
4. **POST /api/v1/user/signup**: Endpoint to register users for the companies.
5. **GET /api/v1/user/resolve**: Endpoint to retrieve all users for a company/AppKey.

Metadata

Metadata are dynamic variable related to an entity. We assumed the entity is a user. The endpoints served by the service are :->

1. **GET /api/v1/ping**: The endpoint is to check the health of the service.
2. **POST /api/v1/users/meta**: Create/Replace User.
3. **PATCH /api/v1/users/{userId}/meta**: Merge the given user metadata with existing value.
4. **GET /api/v1/users**: Returns all the users page by page.
5. **POST /api/v1/users/meta/search**: Gets users satisfying the query. There are three supported types of queries available. **MATCH**, **EXCEPT**, and **RANGE**.

MATCH

Match query takes the value as key, value pairs.

```
"match": {  
  "key1": "value1",  
  "key2": ["value2", "value3"]  
}
```

Keys are the field name of the JSON in the metadata column. For a match query, every key must satisfy the value. The values are usually objects of primitive types or arrays of primitive types. If values are of primitive types then the JSON fields are just checked if they are matched.

If they are arrays then the corresponding JSON field must also be an array. If not then they will not be in the result. And also all the items in the array must be available in the JSON field.

It is also possible to search if at least one item in the array is matched. Then this must be specified using an 'ANY' clause. By default, it is assumed as "ALL" clause.

```
"match": {  
  "carrying": {  
    "ANY": ["abul", "mofiz"]  
  }  
}
```

```
}
```

ANY

The value of "ANY" must be a list type. Satisfies if any of the item of the list is found.

```
"match": {  
  "carrying": {  
    "ANY": ["abul", "mofiz"]  
  }  
}
```

ALL

The value of "ALL" must be a list type. Satisfies if all of the items on the list is found. This is the default behaviour if ALL or ANY nothing is specified for an array.

```
"match": {  
  "carrying": {  
    "ALL": ["abul", "mofiz"]  
  }  
}
```

EXCEPT

Except query takes the value as key, value pairs just like the match query.

```
"except": {  
  "key1": "value1",  
  "key2": ["value2", "value3"]  
}
```

For an except query every key must not satisfies the value. It is just the opposite of a match. If they match then they will be removed from the result in an except query.

Except query can also contain "ANY" or "ALL" clause. Default is "ALL" if nothing is specified.

Gateway

The service is the Gateway of all services. All public endpoints have to access through the gateway. Restricted endpoints can not be accessed through the gateway, they can be accessed another way. A custom health checker named **rls_auth** is used here. Any request to the gateway is first checked by the plugin, if the user has access to the endpoint then the request is forwarded to the

Cloud Solution

The whole infrastructure is deployed on AWS.

IAC

IAC stands for Infrastructure as code. Terraform is used here as an IAC tool. The whole infrastructure is written as terraform configuration for provisioning AWS resources. When we have to deal with a large infrastructure it is very handy to use an IAC tool. Among lots of IAC tools, we choose to use terraform because it supports all the Cloud Providers and as we just need an IAC tool for provisioning resources terraform is best fitted to our requirement. The whole infrastructure is divided into three sections:

1. Base infrastructure.
2. Platform
3. Application

Base Infrastructure

In this section terraform configurations are written for

1. Redis.
2. EC2 instance for Rabbitmq.
3. Mysql database.
4. EC2 instance for Consul.
5. ECR repository for storing Ping, History, Id, Metadata, and Gateway images.
- 6.

Platform

Configuration written in this section are :->

1. Autoscaling group, launch configuration, and cluster for **Ping** service.
2. Autoscaling group, launch configuration, and cluster for **History** service.
3. Autoscaling group, launch configuration, and cluster for **Id** service.
4. Autoscaling group, launch configuration, and cluster for **Metadata** service.
5. Autoscaling group, launch configuration for **Gateway**.

Application

IAC Service-wise

Before deploying the services Mysql database, Redis, Rabbitmq, and Consul instances are required.

Database

Mysql database is deployed on AWS RDS.

- engine version -> 8.0.21

Redis

Redis instance is deployed in AWS using Amazon Elastic Cache. Properties of the Redis instance

- engine -> 5.0.6

Rabbitmq

Rabbitmq is deployed on AWS EC2 instance. The instance type of EC2 instance is **t3.small**.

Consul

For managing service configuration a consul is deployed on **t3.nano** type AWS EC2 instance.

Ping

To deploy Ping service infrastructure used:

- AWS autoscaling group
- Launch configuration
- Cluster
- ECS service
- AWS ECR to store docker image
- AWS service discovery

History

To deploy history service infrastructure used:

- AWS autoscaling group
- Launch configuration
- Cluster
- ECS service
- AWS ECR to store docker image
- AWS service discovery

Id

To deploy id service infrastructure used:

- AWS autoscaling group
- Launch configuration
- Cluster
- ECS service
- AWS ECR to store docker image
- AWS service discovery

Metadata

To deploy metadata service infrastructure used:

- AWS autoscaling group
- Launch configuration
- Cluster
- ECS service
- AWS ECR to store docker image
- AWS service discovery

Gateway

Gateway service is deployed using

- AWS autoscaling group
- Launch configuration
- AWS ECR to store docker image

Load test statistics

Kubernetes master and slave resources

Master

- = c5.xlarge(1)
- Master= vcpu 4
- memory 8gb,

slave

- c5.xlarge(2)
- slave= slave count 8
- cpu: 800m
- memory: 1500M

RPS	Req /s got						Comment
	HTTP code	Highest rate	Mean rate	Actual rate	Total count	%	
3000	200	385.14 / s	910.39 / s	963.256	577954	100%	ping: 1 t3.nano redis: cache.t3.small rabbitmq t3.small The server dies after a while data: 545801
	502	0.7 / s	0.01 / s		7		
	503	4002 / s	805.27 / s		1014639		
	504	382.2 / s	3.03 / s		3822		

1000	200	1252.7 / s	956.96 / s	980.33	588200	100%	ping: 1 t3.nano redis: cache.t3.small rabbitmq t3.small The server can serve properly data: 588200 No data missing
2000	200	2170.3 / s	1933.40 / s	1978.33 /s	1187000	100%	ping: 1 t3.nano Redis: cache.t3.small rabbitmq t3.small The server can serve properly data: 1120265 Data missing = 66735
2000	200	2240.2 / s	1852.20 / s		1135479		ping: 1 t3.nano Redis: cache.t3.small rabbitmq t3.small The server dies at the last moment data: 1120265 Data missing = 2781
	502	62 / s	20.67 / s		620		
	503	2554.2 / s	1530.03 / s		45901		
2000	200	2252.1 / s	1730.67 / s		1180722		ping: 1 t3.nano Redis: cache.t3.small rabbitmq t3.small The server can serve properly Actual data: 1120265 Data missing = 2781
	503	96.3 / s	1.97 / s		963		
	504	91.4 / s	1.87 / s		915		

With a single t3.nano instance, it is possible to serve 1500 requests per second. We have tested the server with 2000 requests per second, but in this case, there are some issues with data missing and the server going down.

